Karpatkey Tokenized Fund KpkShares

Côme du Crest

2025-11-06

Table of contents

- Table of contents
- KpkShares Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med] Performance fees can be disabled through external calculatePerformanceFee
  - [Low] Removing a deposit asset may break accounting
  - Optimisations and miscellaneous

KpkShares Audit

This document presents the finding of a smart contract audit conducted by Côme du Crest for Gnosis.

Scope

The scope includes selected contracts within `karpatkey/karpatkey-tokenized-fund/contracts/src/fund` as of commit `0xdf62b72`.

The code has already been audited and only the updated `KpkShares` contract and its dependencies have been reviewed.

Context

`KpkShares` is a managed subscription contract. Users can request subscription by providing assets in order to receive shares and request redemption by providing shares in order to exit assets. The expected amount of shares / assets to receive is provided by the user during request. The effective amount of shares / assets received during subscription / redeem is computed from the shares price denominated in assets and cannot be lower than the value expected by the user.

Assets and shares are held in escrow in `KpkShares` for pending subscriptions / redemptions. Otherwise, received assets are sent to a portfolio safe for re-investment.

The protocol implements multiple fees: a performance fee charged on an increase of the shares price in USD, a management fee charged over time as new minted shares, and a redemption fee charged as a portion of the redeemed shares.

The protocol relies on a complete trust of the admin that can upgrade the `KpkShares` contract and the operator that can charge any amount of fees they like.

Status

The report has been sent to the core developer.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Karpatkey (the “Company”), with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Med] Performance fees can be disabled through external calculatePerformanceFee()

Summary

The contract `WatermarkFee` exposes the external function `calculatePerformanceFee()` called by `KpkShares` to calculate the performance fee based on the current shares price. This function can be called by anyone to update the storage variable `highWatermark` that will break the accounting of the previous highest price of the shares thus breaking the calculation of the performance fee.

Vulnerability Detail

The function `calculatePerformanceFee()` can be called by anyone with a value of `sharesPrice = 2**256 - 1` to set `highWatermark` to this max value. Any subsequent call to `calculatePerformanceFee()` will return 0, effectively removing performance fees:

```
contract WatermarkFee is IPerfFeeModule {
    /// @notice The current high watermark (highest share price seen)
    uint256 public highWatermark;
    ...

    function calculatePerformanceFee(uint256 sharesPrice, uint256 timeElapsed,
        uint256 feePct)
        external
        returns (uint256 fee)
    {
        uint256 previousWatermark = highWatermark;
        ...
        if (sharesPrice <= previousWatermark) {
            return 0;
        } else {
            // Only update and emit if the watermark actually changes
            if (highWatermark != sharesPrice) {
                highWatermark = sharesPrice;
                emit WatermarkUpdated(previousWatermark, highWatermark);
            }
        }
        ...
    }
}
```

Impact

Performance fees can be deactivated by anyone.

Code Snippets

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/FeeModules/WatermarkFee.sol#L33

Recommendation

Store the `highWatermark` value in `KpkShares` instead of `WatermarkFee` and pass it as argument to `calculatePerformanceFee()`. This will additionally avoid potential errors when one `IPerfFeeModule` is switched to another one in `KpkShares` since the `highWatermark` storage value would need to be synced between the old and new `IPerfFeeModule`.

Otherwise, add access control so that `WatermarkFee.calculatePerformanceFee()` can only be called by `KpkShares`.

[Low] Removing a deposit asset may break accounting

Summary

The function `KpkShares.updateAsset()` can be used by an operator to remove a deposit asset. If an asset is no longer used as deposit nor withdraw, the key `_approvedAssetsMap[asset]` will be deleted.

As a result the internal function that computes the recoverable amount `_assetRecoverableAmount()` will not take into account the amount of pending subscription tokens and allow sending the total contract balance to the portfolio safe.

This will not leave any token in the contract for users to cancel their pending requests.

Vulnerability Detail

The function `_updateAsset()` will delete `_approvedAssetsMap[asset]` if the asset is totally removed:

```
    function _updateAsset(address asset, bool isUsd, bool canDeposit, bool
        canRedeem) internal {
        if (asset == address(0)) revert InvalidArguments();

        //check if asset exists
        if (_approvedAssetsMap[asset].asset != address(0)) {
            //check if asset is being removed
            if (!canDeposit && !canRedeem) {
                _shadowAsset(asset);
                delete _approvedAssetsMap[asset];
                emit AssetRemove(asset);
            } else {
                ...
            }
        } else {
            ...
        }
    }
```

The function `_assetRecoverableAmount()` will not take into account pending `subscriptionAssets[token]` if the asset has been deleted in the mapping:

```
    function _assetRecoverableAmount(address token) internal view override(
        RecoverFunds) returns (uint256) {
        if (_approvedAssetsMap[token].asset != address(0)) {
            return IERC20(token).balanceOf(address(this)) - subscriptionAssets[
                token];
        }
        if (token == address(this)) {
            return 0;
```

```
        }
        return super._assetRecoverableAmount(token);
    }
```

Anyone can call `recoverAssets()` on that asset which will prevent the pending subscription requests to be cancelled due to not having enough tokens in escrow:

```
    function recoverAssets(address[] calldata assets) external {
        uint256 length = assets.length;
        for (uint256 i; i < length; ++i) {
            IERC20(assets[i]).safeTransfer(_assetRecoverer(),
                _assetRecoverableAmount(assets[i]));
        }
    }
```

Impact

If assets are disabled in `KpkShares` while subscriptions are pending, they may get stuck if assets are recovered to the portfolio safe. This may happen accidentally when portfolio managers withdraw tokens from `KpkShares` for regular operations.

Code Snippets

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L554-L562

Recommendation

Remove `subscriptionAssets[token]` from the recoverable amount even when the asset has been removed:

```
    function _assetRecoverableAmount(address token) internal view override(
        RecoverFunds) returns (uint256) {
        if (token == address(this)) {
            return 0;
        }
        return super._assetRecoverableAmount(token) - subscriptionAssets[token
            ];
    }
```

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments. These comments can be safely acknowledged.

Gas saving by reordering ApprovedAsset struct

The struct `ApprovedAsset` can be re-ordered to move `string symbol` at the end of it to save one storage slot. The address, uint8, and bools will all be stored in the same slot hence bringing down the slot count to 2 instead of 3:

```
    struct ApprovedAsset {
        address asset;
        uint8 decimals;
        bool isUsd;
        bool canDeposit;
        bool canRedeem;
        string symbol;
    }
```

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/IkpkShares.sol#L79-L86

Minimum time elapsed for fee lower than documented

The documentation states that the minimum time elapsed for fee calculation should be 1 day but the contract uses a constant of 6 hours.

```
contract KpkShares is
    ...
{
    ...
    uint256 public constant MIN_TIME_ELAPSED = 6 hours;
    ...
}
```

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L55

Default admin role can create new admins

The documentation states that the default admin role `can grant/revoke other roles` but it is its own admin so it can also grant/revoke the admin role to other addresses.

Unclear decimals used in sharesPrice

The functions `assetsToShares()` and `sharesToAssets()` use an input `sharesPrice` to convert in between assets and shares. This `sharesPrice` is documented as "the current price per share in asset units", which can be interpreted as using the underlying asset decimals. However, the price should be provided using 8 decimals.

I feel like this should be documented better in the code.

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/IkpkShares.sol#L309

_validateInitializationParams() does not validate performanceFeeModule

The performance fee module can be set to the zero address in `initialize()` since `_validateInitializationP()` does not enforce checks on it:

```
1     function _validateInitializationParams(ConstructorParams memory params)
          internal pure {
2         if (
3             params.asset == address(0) || params.admin == address(0) || params.
                  safe == address(0)
4                 || params.feeReceiver == address(0) || params.
                      subscriptionRequestTtl == 0
5                 || params.redemptionRequestTtl == 0
6         ) {
7             revert InvalidArguments();
8         }
9
10        // Validate fee rates are within reasonable bounds.
11        if (params.managementFeeRate > MAX_FEE_RATE) revert
              FeeRateLimitExceeded();
12        if (params.performanceFeeRate > MAX_FEE_RATE) revert
              FeeRateLimitExceeded();
13        if (params.redemptionFeeRate > MAX_FEE_RATE) revert
              FeeRateLimitExceeded();
14    }
```

This is inconsistent with `setPerformanceFeeModule()` which prevents setting the fee module to `address(0)`:

```
1     function setPerformanceFeeModule(address newPerfFeeModule) external isAdmin
          {
2         if (newPerfFeeModule == address(0)) revert InvalidArguments();
3
4         _setPerformanceFeeModule(newPerfFeeModule);
5     }
```

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L579

Performance fee only charged when asset is USD

The function `_chargeFees()` only charges performance fee when the asset used to process requests is a USD stablecoin. This makes sense due to the fact that performance fee should track the highest value of the shares in USD and only charge fees when it is increased. However, this behaviour is undocumented and could result in the contract under-charging fees if requests are processed in non-USD assets before performance fee can be charged.

```
1     function _chargeFees(address asset, uint256 sharesPriceInAsset) internal
          returns (FeesCharged memory fees) {
2         uint256 timeElapsed = block.timestamp - lastUpdate;
3         if (timeElapsed > MIN_TIME_ELAPSED) {
4             ...
5             if (performanceFeeRate > 0 && _approvedAssetsMap[asset].isUsd) {
6                 fees.performanceFee = _chargePerformanceFee(sharesPriceInAsset,
                      timeElapsed);
7             }
8             ...
9         }
10        return fees;
11    }
```

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L793

Unused variables

The contract `WatermarkFee` declares an unused storage variable `lastTotalSupply` that should be removed.

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/FeeModules/WatermarkFee.sol#L15

The contract `KpkShares` declares an unused constant `_NORMALIZED_PRECISION_USD` that could be used in shares conversion formulas. It also has the `updateRequestTtl` storage variable that can be removed.

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L508

https://github.com/karpatkey/karpatkey-tokenized-fund/blob/df62b7238a4a337766adda058bba765ac89071df/contracts/src/fund/kpkShares.sol#L89